

포인터의 기본은 변수의 개념에서 출발한다.

변수는 특정 메모리를 할당 받아 그 메모리 안에 데이터를 넣는 것으로, 실제 많은 프로그래밍 언어에서는 객체, 배열, 리스트 같은 데이터를 변수에 담을 때 데이터의 실제 값을 저장하지 않는다. 데이터가 저장된 메모리의 주소값을 변수에 저장하게 되며 변수를 호출하면 주소에 찾아가 실제 데이터가 있는 곳에서 값을 가져오는 것이다.

따라서 데이터의 주소(위치)를 참조하는 경우에는 여러 변수가 한 가지 주소를 가리키고 있다면 한 변수에서 데이터를 수정할 시 같은 주소를 참조하는 다른 변수의 데이터도 같이 바뀐다. 하지만 숫자나 불리언 같은 데이터의 경우에는 포인터를 사용하지 않는다면 값을 '복사'해서 가져온다. 따라서 한쪽을 수정해도 서로 영향을 주지 않게 되는 것이다.

또 메모리 주소를 담는 변수라는 점에서 포인터를 완벽하게 이해하려면 운영체제가 프로그램이 실행할 때 어떻게 메모리 구조를 다루는지를 알아야한다.

Text(Code) 영역은 작성한 기계어 코드가 올라가는 공간이다. 함수 포인터가 가리키는 주소가 이 영역에 존재하는 것이다. Data / VSS 영역에 전역 변수와 정적(static) 변수가 저장된다. Heap 영역은 malloc() 등으로 동적 할당되는 공간으로, 런타임의 크기가 결정되는 데이터를 담을 때 이 영역의 메모리를 제어한다. Stack 영역은 지역 변수와 매개변수가 저장된다. 우리가 흔히 선언하는 포인터 변수 (int *p;) 자체도 함수 내부에 있다면 이 Stack 영역에 존재하면서, Heap이나 Data 등 다른 영역의 주소를 담게 되는 것이다. (따라서 포인터 변수 자체는 지역 변수일 때 Stack 영역에 존재하는 것이지만 변수가 가리키는 데이터 값이 다른 Low한 영역의 주소 등을 담아 쉽게 다룰 수 있게 하는 것이다)

배열과 포인터는 매우 유사한 개념이지만 본질적으로는 다르다. 배열이 수식 내에서 사용될 때 배열의 첫 번째 요소를 가리키는 포인터로 Decay되는 점에서 포인터와 매우 유사하다고 할 수 있다 말하는 것이다.

하지만 배열은 메모리 상에 연속적으로 할당된 데이터 묶음 자체이고, 포인터는 다른 메모리 공간의 주소값을 담는 단일 변수다. 크기도 다르고, 배열 자체의 주소를 다루는 것은 불가능하기까지 하다. 포인터는 다른 변수나 주소를 가리키도록 변경이 가능하다.(물론 어떤 변수 자체의 주소를 변경하기 위한 생각을 했다면 이중 포인터를 써야될 때가 온 것이겠지만..)

따라서 실무나 시스템 레벨에서의 포인터 제어에는 단지 주소만 가리키는 포인터만 가지고는 자유롭게 데이터를 다루기 힘들 수 있다.

따라서 포인터 변수 자체의 주소를 담는 변수인 이중 포인터의 개념을 사용해서 함수 내부에서 외부 포인터가 가리키는 대상 자체를 바꾼다. 이중 포인터는 대표적으로 2차원 배열을 동적 할당하거나, 연결 리스트에서 노드를 삽입하거나 삭제할 때 포인터의 주소를 넘겨주는 상황일 때 등장한다.

보이드 포인터는 데이터의 자료형을 명시하지 않는 일반화된 포인터로 어떤 타입의 주소든 담을 수 있지만 가리키는 대상의 크기를 모르기 때문에 역참조나 포인터 연산이 불가능하다. 사용할 때는 반드시 원하는 타입으로 캐스팅을 해야하며 대표적으로 malloc의 반환형이 void *인 것을 알면 이해가 쉬울 것이다.

volatile 키워드는 특정 메모리 주소를 포인터로 접근할 때 컴파일러의 최적화를 막는 키워드다. 메모리에 매핑된 하드웨어 레지스터를 제어할 때 필수적인 개념이다.